

GridBasedSandbox

Project State Summary & Future Implementation Roadmap

Project Concept: A hybrid Minecraft-style voxel sandbox + grid-based robotics & automation game developed in Unity. The engine integrates an in-game Python environment (**PyBridge** via IronPython), enabling players to explore infinite voxel worlds, construct complex structures, and deploy programmable autonomous robots and sensor peripherals communicating across an event bus.

1. Current State — Implemented Subsystems

Voxel World & Chunk Engine

- **3D Chunk Streaming:** Infinite-ish chunk management loaded dynamically around the player (*VoxelWorldManager*, *VoxelChunk*).
- **Greedy Face Culling:** Optimized mesh generation discarding hidden and internal faces with cross-chunk border sampling (*VoxelMeshBuilder*).
- **Layered Noise & Biomes:** Multi-octave Perlin / fBm noise heightmap generation with ScriptableObject biome configurations (*BiomeDefinition*, *OverworldGenerator*).
- **Budgeted Meshing:** Frame-budgeted chunk rebuild queue (*maxChunkBuildsPerFrame*) preventing rendering hitches and stuttering.

PyBridge — Robotics & In-Game Python Scripting

- **Embedded IronPython:** Thread-safe background execution runtime isolated from main thread Unity crashes (*ScriptRunner*).
- **Robotic Locomotion:** Discrete grid stepping (*move*, *move_back*), 90-degree yaw rotation (*turn_left*, *turn_right*), and speed adjustments (*VoxelGridMotor*).
- **Vertical Agility & Jump Physics:** In-place hops (*jump*), directional climbing (*jump_forward*, *jump_back*), and momentum-preserving cannonball flight mode.
- **Spatial Voxel Sensors:** Instantaneous obstacle & headroom detection without expensive raycasting (*is_blocked_**, *surroundings*, *can_jump_ahead*).
- **Device Bus Architecture:** Publish/subscribe inter-device messaging (direct messaging, topic filtering, broadcasts, shared status board) (*DeviceBus*, *BaseDeviceAPI*).
- **In-Game IDE:** Interactive code editor modal (`` / ~` hotkey or cursor interaction on robots) with live execution controls and string sanitization (*InGameIDEController*).

First-Person Player Physics & Controller

- **Custom Rigidbody Dynamics:** Snappy Minecraft-like movement without floatiness (*fallGravityMultiplier*, terminal velocity clamp) (*PlayerController*).
- **Collision Precision:** Fits seamlessly through 1x2 voxel gaps, prevents pillar-floating, edge-sticking, and physics jitter.
- **State-Aware Input:** Seamlessly releases cursor lock and suppresses gameplay controls while the IDE is active (*GameState*).

Grid Placement & World Interaction

- **Dual Placement Pipeline:** Unified system handling both block modifications (*VoxelBlockPlacer*) and interactive entity placement (*SimpleObjectPlacer*).
- **Terrain Collision Guard:** Strict validation ensuring placed machines cannot intersect solid voxel terrain cells.
- **Interaction Layer:** Empty-hand context-sensitive interaction (*Interactable*) for activating switches or accessing device terminals.

Modular Inventory & Persistence Architecture

- **Separation of Concerns:** Strict decoupling between in-memory slots (*InventoryManager*), hotbar presentation (*InventoryUI*), and loader initialization.
- **Priority Grant Pipeline:** Dynamic resolution picking between saved game data (priority 0), level grants (priority 50), and default loadouts (priority 100).
- **Persistence:** JSON-based serialization into PlayerPrefs for hotbar stacks, items, and quantities (*InventorySaveService*).

2. Strategic Roadmap & Next Implementations

A. Robotics & Industrial Automation (Key Differentiator)

- **Robot Inventory & Block Manipulation:** Equip robots with internal inventory slots and Python APIs: `robot.mine_ahead()`, `robot.mine_below()`, `robot.place_block('stone')` to enable fully automated tunneling, bridge-building, and mining.
- **Peripherals & Sensors:** Implement programmable pressure plates, tripwires, and laser trip sensors that publish trigger events onto the DeviceBus.
- **Conveyor Belts & Hoppers:** Build material transport pipes/belts allowing automated drop-off from robot inventories into storage chests.
- **Script Library & Presets:** Allow players to save custom Python scripts to disk (.py files) with an in-game IDE preset loader (e.g. *quarry.py*, *patrol.py*, *follow.py*).

B. Inventory, Crafting & Survival Progression

- **Expanded Grid Inventory:** Extend beyond the 10-slot hotbar to a full 27-slot bag view and craftable storage chests.
- **Item Pickups & Drops:** Implement item dropping (Q key) and physical floating voxel item pickups with magnetic attraction.
- **Workbenches & Assembly Recipes:** Craft robot components, sensor modules, batteries, logic chips, and mining tools.

C. World Generation & Environmental Dynamics

- **3D Cave Systems:** Integrate a 3D simplex noise carving pass into *OverworldGenerator* to spawn caverns and subterranean ore veins.
- **Specialized Voxel Materials:** Add transparent culling for Glass/Ice, and gravity-affected blocks (Sand, Gravel).
- **Procedural Structures:** Spawn ruins, abandoned bunkers, and research labs containing rare script cartridges and components.

D. Performance & Engineering Optimizations

- **Background Threaded Meshing:** Migrate *VoxelMeshBuilder.Build()* to background worker tasks / Unity Job System, keeping only mesh application on the main thread.
- **Greedy Face Merging:** Merge coplanar adjacent quad faces of identical block types, cutting chunk vertex and triangle counts by up to 70%.
- **Persistent World Saves:** Serialize modified dirty chunks into compressed regional save files.